

SCRIPTS GUIDE

version0_15 Scripts README

실험 진입점, 자동 후처리, DET 점수 체계, failure reason 해석을 한 번에 보는 시각 가이드

Generated at 2026-05-06 11:48

처음 시작

스크립트 지도

실행 시나리오

결과 구조

DET 설명

failure_reasons

실제 사례

디버깅 순서

이름 매핑

가장 먼저 기억할 것

`run_benchmark.py` 하나면 `benchmark` 흐름은 거의 시작할 수 있습니다.

QUICK ORIENTATION

복잡한 스크립트를 실행 순서 기준으로 다시 정리한 문서

이 문서는 version0_15_update20260413/scripts/ 안의 스크립트들을 “무엇을 먼저 실행해야 하는지”, “무슨 결과가 자동으로 생기는지”, “DET와 failure reasons를 어떻게 읽어야 하는지” 기준으로 다시 구조화한 가이드입니다.

가장 먼저 실행할 파일

run_benchmark.py 하나만 기억하면 됩니다.

single-model이면 row report가 자동으로 붙고, 필요한 비교와 audit도 옵션으로 이어집니다.

실험 비교

--compare-to를 주면 baseline vs retrieval 같은 A/B 비교를 자동으로 수행합니다.

별도 실행은 **compare_benchmarks.py**입니다.

DET 읽는 기준

역사 비교와 GA는 **legacy**, 실제 품질 판정과 리포트는 **strict**가 권장됩니다.

가장 흔한 오해

failure_reasons는 런타임 크래시 로그가 아니라 **왜 감점됐는지**를 설명하는 태그입니다.

START-HERE

처음 시작할 때 보는 순서

복잡한 스크립트들을 한 번에 외우기보다, 가장 짧은 의사결정 경로만 먼저 잡는 것이 좋습니다.

1. 환경 준비

BASH

```
cd /home/mgjeong/Desktop/llm/JOILang-Server

export JOI_V15_WORKER_PYTHON=/home/mgjeong/miniconda3/envs/l/bin/python
export JOI_V15_LOCAL_DEVICE=cuda:0
```

2. 모델 준비 상태 확인

BASH

```
python gpt_mg/version0_15_update20260413/scripts/run_benchmark.py --suite |
```

3. 가장 기본적인 smoke test

BASH

```
python gpt_mg/version0_15_update20260413/scripts/run_benchmark.py --suite |
```

4. 자주 쓰는 후처리 옵션

- `--analyze-det`: legacy vs strict DET audit
- `--compare-to /abs/path`: 이전 실험과 자동 비교
- `--export-row-report`: multi-model run에서도 row report 생성
- `--export-paper-artifacts`: figure / table export

SCRIPT-MAP

스크립트 지도

이 폴더 안 파일이 많아 보여도, 실제로는 상위 진입점 몇 개와 내부용 스크립트로 나뉩니다.

파일	분류	언제 쓰는가
run_benchmark.py	권장	메인 benchmark 실행기. 실험 진입점으로 가장 역할 파일.
compare_benchmarks.py	직접 실행	baseline vs retrieval 등 두 결과 디렉터리 비교
export_row_report.py	직접 실행	GT JOIcode vs Generated JOIcode를 좌우 HTML/PDF 리포트 생성.
audit_det.py	직접 실행	legacy vs strict DET 차이 분석 및 과대평가 탐
export_paper_figures.py	직접 실행	scatter / bar / category figure export.
prepare_local_models.py	준비	local model cache, readiness, preflight 준비
inspect_service_context.py	디버깅	schema fallback vs retrieval fallback prom 구성 확인.
run_generate.py	내부용	candidate generation 저수준 실행기. 보통은 지 않음.
run_rerank.py	내부용	DET rerank / repair 저수준 실행기.
run_ga_search.py	실험	GA prompt search.
run_category_sweep.py	실험	category sweep.
run_admin_feedback_update.py	실험	admin feedback update pipeline.

실무 권장: 일반 사용자는 `run_generate.py`, `run_rerank.py`를 직접 치기보다 `run_benchmark.py`로 먼저 시작하는 편이 안전합니다.

WORKFLOWS

가장 자주 쓰는 실행 시나리오

복사해서 바로 쓸 수 있는 대표 패턴을 시나리오별로 정리했습니다.

단일 모델 + 자동 row report

BASH

```
python gpt_mg/version0_15_update20260413/scripts/run_benchmark.py --suite |
```

single-model run이면 기본적으로 row report HTML/PDF가 자동 생성됩니다.

단일 모델 + DET audit

BASH

```
python gpt_mg/version0_15_update20260413/scripts/run_benchmark.py --suite |
```

기존 run과 자동 비교

BASH

```
python gpt_mg/version0_15_update20260413/scripts/run_benchmark.py --suite |
```

multi-model run + 특정 모델 row report

BASH

```
python gpt_mg/version0_15_update20260413/scripts/run_benchmark.py --suite |
```

RESULTS

benchmark가 끝나면 어떤 결과가 생기나

실험 결과는 보통 `results/model_suite_<timestamp>/` 아래에 쌓입니다. 무엇을 먼저 봐야 하는지도 같이 표시했습니다.

핵심 CSV

- `suite_summary.csv`: 모델 단위 요약
- `row_comparison.csv`: row 단위 상세 결과
- `failure_reason_summary.csv`: failure reason 집계
- `category_summary.csv`: category 단위 요약

자동 후처리 산출물

- `side_by_side_report_<model_key>/report.html`
- `side_by_side_report_<model_key>/report.pdf`
- `det_audit/analysis_summary.json`
- `context_compare_.../context_comparison_summary.json`

논문용 산출물

- `main_model_comparison.csv`
- `tradeoff_summary.csv`
- `paper_metrics_summary.json`
- `paper_figures/...`

DET

DET를 어떻게 읽어야 하나

DET는 단순 exact match가 아니라, service 해석·의미 정렬·GT coverage·값 흐름을 여러 축으로 합산한 점수입니다.

프로파일	장점	약점	권장 용도
legacy	기존 실험과 호환, 빠르고 비교적 관대함	값 흐름, receiver coverage, numeric/enum grounding을 후하게 줄 수 있음	historical comparison search
strict	GT service coverage, receiver coverage, dataflow, numeric/enum grounding을 엄격히 반영	service equivalence가 없는 케이스는 다소 보수적으로 볼 수 있음	최종 품질 리포트, 논문

권장 해석: historical/GA 비교는 legacy, 실제 품질 리포트와 acceptance는 strict를 우선으로 보세요.

det_score

최종 DET 점수. 0~100.

det_gt_exact

GT와 exact match 여부.

det_gt_similarity

GT와의 전체 유사도. exact가 아니어도 구조가 비슷하면 일부 점수를 받습니다.

det_service_match

생성 코드의 서비스가 schema에 얼마나 잘 resolve되는지.

det_arg_type_ok

각 서비스의 인자 타입과 개수가 schema expectation과 얼마나 맞는지.

det_precondition_ok

전제조건이나 guard가 필요한 경우 그것이 반영됐는지.

det_semantic_ok

명령 의미와 action 방향이 얼마나 맞는지.

det_min_extraneous

명령에 없는 불필요한 action이 섞였는지.

det_gt_service_coverage

GT에 필요한 서비스를 생성 코드가 모두 포함했는지.

det_gt_receiver_coverage

GT의 receiver scope와 instance coverage를 얼마나 잘 덮었는지.

det_dataflow_score

sensor/value를 읽은 뒤 실제 downstream 함수에 전달했는지.

det_numeric_grounding

시간, 채널, 온도 같은 숫자 정보가 정확히 반영됐는지.

det_enum_grounding

enum 값(dry, emergency 등)이 정확한지.

FAILURE-REASONS

failure_reasons 상세 해설

failure_reasons는 런타임 크래시 로그가 아니라, 왜 감점됐는지를 설명하는 진단 태그입니다.

FAILURE_REASON	무슨 뜻인가	대표 원인	읽는 팁
invalid_json	출력이 JSON으로 파싱되지 않음	자유 텍스트, braces mismatch, explanation 혼입	맨 앞단 머리 지킴 저 해결
schema_missing_keys	JSON은 맞지만 name / cron / period / code 중 필수 키 누락	code 없이 script만 반환	출력 스크립트 확인
no_parseable_member_access	code는 있지만 (#Device).Service(...) 형식으로 parse 불가	JOILang 대신 의사코드 반환	코드 포맷 고정
unknown_service:<member>	schema에서 매핑 불가능한 service/member	invented helper, 심한 오타, 잘못된 camel/snake case	schema ground
service_match	전체 호출 중 일부가 schema에 정확히 resolve되지 않음	맞는 서비스와 틀린 서비스 혼재	unknown service! 자주 나타
arg_type	서비스는 맞지만 인자 타입/개수/순서 불일치	숫자 대신 string, enum 대신 자유 텍스트	numerical ground 이 해석
precondition	필요한 전제조건/guard가 부족	power/switch guard 없이 바로 action	명령 의미 행 안전성
semantic	명령 의미와 action 방향이 약하게 맞음	increase/decrease, lock/unlock 반대	service 맞아도 굿날 수
extraneous	명령에 없는 불필요한 action 추가	speaker가 필요 없는 데 speaker까지 사용	과생성

FAILURE_REASON	무슨 뜻인가	대표 원인	읽는 팁
gt_mismatch	GT와 exact match는 아님	매우 넓은 요약 태그	이것만 보 아래 cov 항목을 곁 함
gt_service_coverage	GT에 필요한 서비스 일 부 누락	HumiditySensor + Speaker 중 Speaker 누락	가장 자 감점 사
gt_receiver_coverage	GT의 receiver scope 일부 누락	Front/Back 둘 다 필 요한데 하나만 처리	all(...) 거 에서 중
dataflow	읽은 값이 실제 sink로 전달되지 않음	sensor value를 읽기 만 하고 speak/set에 안 씀	strict에
numeric_grounding	숫자/단위 정보가 정확 하지 않음	30분을 1800초 대신 ms 표현 또는 잘못된 수치	strict에 감점
enum_grounding	enum 값이 틀림	dry 대신 warm	strict에 감점

가장 흔한 오해: gt_mismatch 하나만 보고 판단하면 원인을 놓치기 쉽습니다.

gt_service_coverage, gt_receiver_coverage, dataflow, numeric_grounding, enum_grounding을 같이 봐야 합니다.

CASES

실제 DET 해석 예시

legacy가 후하게 준 케이스와 strict가 더 정확하게 본 케이스를 같이 보면, 어떤 failure를 어떻게 읽어야 하는지 훨씬 빠르게 익힐 수 있습니다.

케이스	LEGACY	STRICT	주요 FAILURE	해석
row 10 humidity - sensor_only	87.7078	67.5942	dataflow, gt_mismatch, gt_receiver_coverage, gt_service_coverage	센서값은 유효하지만 실제로 sp... 지 전달하... 드. legacy... 고 strict는... 흐름이 끊... 있습니다.
row 10 humidity - malformed_chain	96.1031	69.9	dataflow, gt_mismatch	겉보기에는... 지만 sour... 가 sink에... 결되지 않... 입니다.
row 3 rice cooker - wrong_unit_ms_expr	93.9479	69.9	arg_type, gt_mismatch, numeric_grounding	시간/단위... 게 틀린 코... strict에서... 내려갑니...
row 37 door lock - explicit_two	92.3641	97.0607	gt_mismatch	GT의 all(. Front/Back... 로 명시적으... 이스. stric... receiver c... 가 충분해... 볼 수 있습...
row 37 door lock - only_back	96.7727	69.9	gt_mismatch, gt_receiver_coverage	그룹 recei... 부만 처리... legacy가... 준 대표 사...

남아 있는 한계: MoveToRGB vs SetColor처럼 service equivalence map이 필요한 케이스는 strict가 다소 보수적으로 볼 수 있습니다.

PLAYBOOK

문제 분석 순서

row report와 CSV를 볼 때 어디서부터 읽으면 빠른지, 실무 기준 디버깅 순서를 정리했습니다.

row report 먼저 볼 때

- DET와 failure_reasons를 먼저 확인
- gt_service_coverage, gt_receiver_coverage, dataflow를 바로 같이 보기
- GT vs Generated JOICode를 좌우 배치로 비교
- 필요하면 prompt_tokens, llm_latency_sec, peak_vram_gb까지 확인

CSV / audit으로 깊게 볼 때

- failure_reason_summary.csv에서 모델별 failure distribution 확인
- row_comparison.csv에서 낮은 DET row 필터링
- audit_det.py로 legacy vs strict 차이 확인
- compare_benchmarks.py로 baseline vs retrieval 변화 확인

실무 팁: legacy high && strict low인 row는 evaluator가 후하게 준 의심 케이스라서 우선 순위 높게 보는 것이 좋습니다.

NAMING

짧은 이름 vs 기존 긴 이름

새 짧은 이름이 권장 entrypoint이고, 기존 긴 이름은 호환용으로 유지됩니다.

기존 이름	권장 이름	설명
run_model_suite_benchmark.py	run_benchmark.py	메인 ben 실행기
compare_service_context_results.py	compare_benchmarks.py	두 결과 디 비교
export_joi_code_side_by_side_report.py	export_row_report.py	GT vs Ge HTML/PI report
analyze_det_profiles.py	audit_det.py	DET prof

Visual companion for README.md · source folder: /home/mgjeong/Desktop/llm/JOILang-Server/gpt_mg/version0_15_update20260413/scripts